

Global Population Forecasting using Machine Learning

Author: Jwel Aktar

Tools: Python, Pandas, NumPy, XGBoost, Streamlit, Git, GitHub

Objective: Analyze historical global population trends and predict future population (2030)

1. Problem Statement

The objective of this project was:

- Analyze historical global population data (1970–2022)
- Identify growth patterns
- Engineer meaningful features
- Build a machine learning regression model
- Predict future population (2030)
- Deploy model as an interactive web application

2. Tools & Technologies Used

Category	Tools Used
Programming	Python
Data Handling	Pandas, NumPy
Visualization	Plotly
ML Model	XGBoost Regressor
Deployment	Streamlit Cloud
Version Control	Git & GitHub

3. Data Understanding

Dataset: `world_population.csv`

Main Variables:

- Country/Territory
- Continent
- Area (km²)
- Growth Rate
- Population (1970–2022)

Type of Data:

- Panel time-series demographic data
- Multi-country longitudinal dataset

4.Data Preprocessing

4.1 Load Data

```
import pandas as pd

df = pd.read_csv("world_population.csv")
```

4.2 Convert Wide → Long Format

Why?

Because ML models work better when each row represents one time observation.

```
population_cols = [
    "1970 Population", "1980 Population", "1990 Population",
    "2000 Population", "2010 Population",
    "2015 Population", "2020 Population", "2022 Population"
]

df_long = df.melt(
    id_vars=["Country/Territory", "Continent", "Area (km²)", "Growth Rate"],
    value_vars=population_cols,
    var_name="Year",
    value_name="Population"
)

# Extract numeric year
df_long["Year"] = df_long["Year"].str.extract(r'(\d+)').astype(int)
```

5. Feature Engineering

5.1 Year Index Feature

```
df_long["Year_Index"] = df_long["Year"] - 1970
```

Purpose:

Convert absolute year into relative time feature.

5.2 Log Transformation (Stabilize Growth)

Population grows exponentially.

```
import numpy as np
```

```
df_long["Log_Population"] = np.log1p(df_long["Population"])
```

Why?

- Reduces skewness
- Makes model more stable
- Improves regression performance

5.3 Continent Encoding

```
df_long["Continent_Encoded"] = df_long["Continent"].astype("category").cat.codes
```

Purpose:

Convert categorical variable into numeric format.

5.4 Lag Feature

Using previous population as predictor:

```
df_long["Lag_Population"] = df_long.groupby("Country/Territory")["Log_Population"].shift(1)
df_long.dropna(inplace=True)
```

Why?

Population is highly dependent on previous year population.

6. Train-Test Split (Time-Aware)

We avoided random split because this is time-series type data.

```
train = df_long[df_long["Year"] <= 2015]
test = df_long[df_long["Year"] > 2015]

features = ["Lag_Population", "Year_Index", "Continent_Encoded", "Area (km²)"]

X_train = train[features]
y_train = train["Log_Population"]

X_test = test[features]
y_test = test["Log_Population"]
```

7. Model Selection

Chosen Model:

XGBoost Regressor

Why?

- Handles non-linear growth patterns
- Works well on tabular data
- Robust to outliers
- High performance

8. Model Training

```
from xgboost import XGBRegressor

model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)
```

9. Model Evaluation

Convert log back to actual scale:

```
from sklearn.metrics import mean_absolute_error, r2_score

y_pred_log = model.predict(X_test)

y_test_actual = np.expml(y_test)
y_pred_actual = np.expml(y_pred_log)

print("MAE:", mean_absolute_error(y_test_actual, y_pred_actual))
print("R2:", r2_score(y_test_actual, y_pred_actual))
```

Metrics Used:

- MAE
- R² Score

10. Model Saving

```
import pickle

with open("model.pkl", "wb") as f:
    pickle.dump(model, f)
```

11.Streamlit Deployment

Built interactive dashboard:

- Country selector
- 2030 prediction
- Animated KPI
- Map visualization
- Country comparison
- Profile branding

Core prediction logic:

```
year_index = 2030 - 1970
lag_population = np.log1p(country_data["2022 Population"].values[0])

features = np.array([[lag_population, year_index, continent_encoded, area]])

prediction_log = model.predict(features)
prediction = np.expml(prediction_log)
```

12.Git & GitHub Version Control

Commands Used:

```
git init
git add .
git commit -m "Initial ML project"
git remote add origin <repo_url>
git push -u origin main
```

13.Streamlit Cloud Deployment

- Connected GitHub repo
- Selected branch main
- Set main file path = app.py
- Deployed live app

Business Impact

This model can help:

- Governments forecast population growth
- Urban planning authorities
- Infrastructure planning
- Resource allocation
- Policy modeling

